

Классификатор обеда

Или как сделать аналог fatsecret

Цель

С помощью Create ML сделать приложение с моделькой, которая **умеет различать разную еду по фото** *(для примера возьмем пиццу, суши и бургеры)*

Шаг 1

Ищем датасет для обучения

Благо существует kaggle и самому фоткать ничего не надо!

Берём отсюда:

- [Pizza, Steak and Sushi Image Dataset](#)
- [Food Recognition - Burger, Pizza & Coke](#)

Pizza, Steak and Sushi Image Dataset

Images of pizza, steak and sushi for classification

[Data Card](#) [Code \(10\)](#) [Discussion \(0\)](#) [Suggestions \(0\)](#)

About Dataset

Dataset Description

This dataset consists of 300 images categorized into three distinct classes: Pizza, Steak, and Sushi. It is divided into two subsets: 225 training images and 75 test images. The images have been collected to facilitate the development and evaluation of image classification models.

Classes:

- Pizza (0)
- Steak (1)
- Sushi (2)

Each image is labeled according to its respective class, making it suitable for training and testing machine learning models for food classification tasks. The dataset can be used for various applications, including deep learning and computer vision projects.

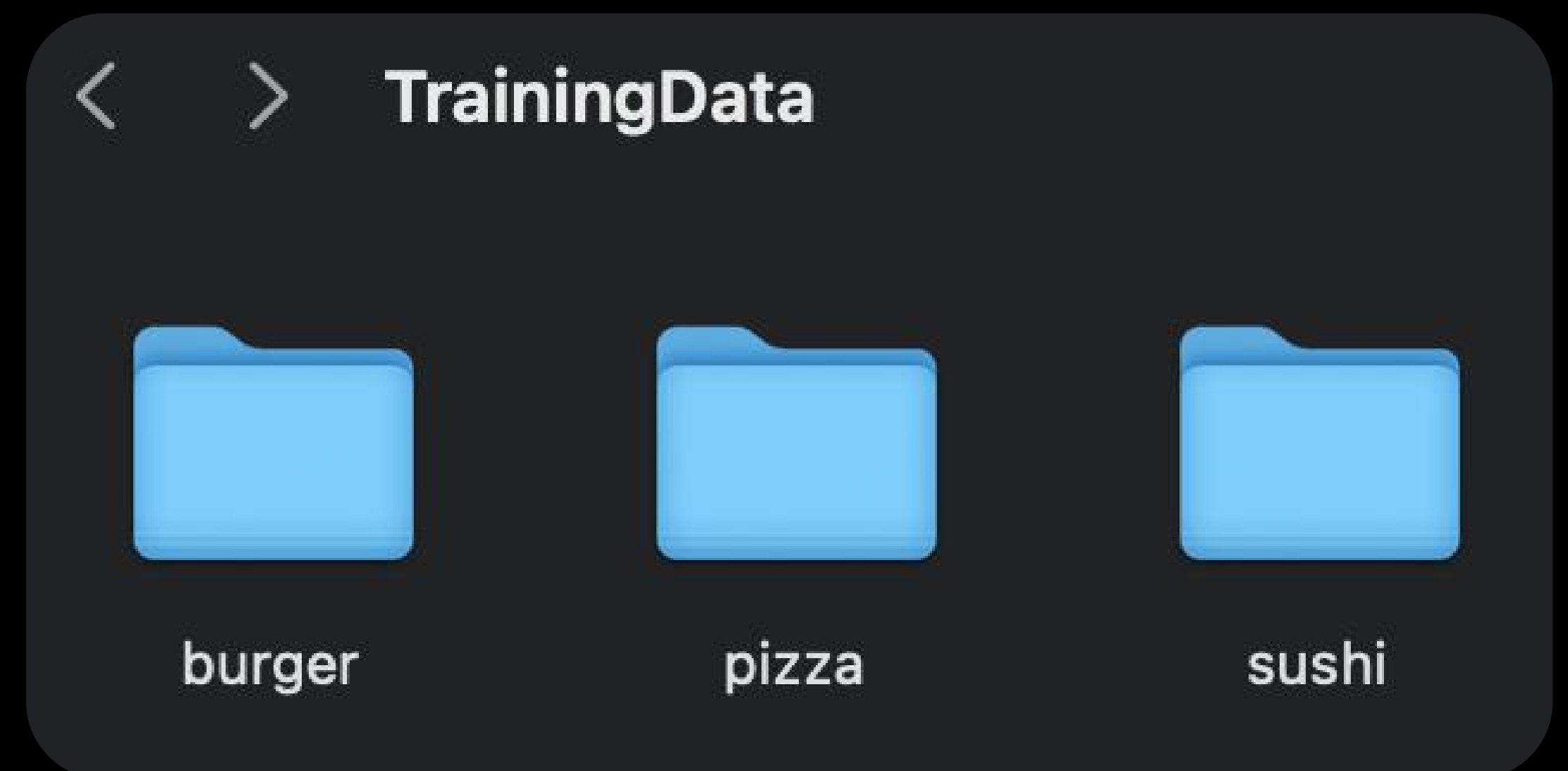
Шаг 2

Подготавливаем данные

Оставляем классы, которые нам нужны:
в данном случае Pizza, Sushi, Burger

Если есть несколько разных датасетов –
объединяем их для лучшей точности нашей
модели

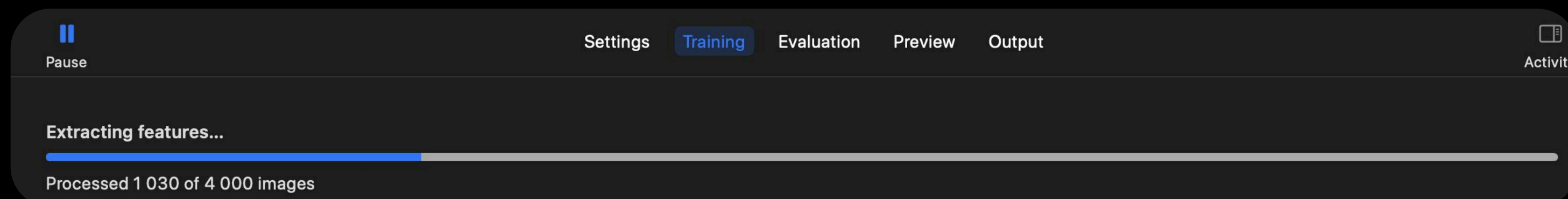
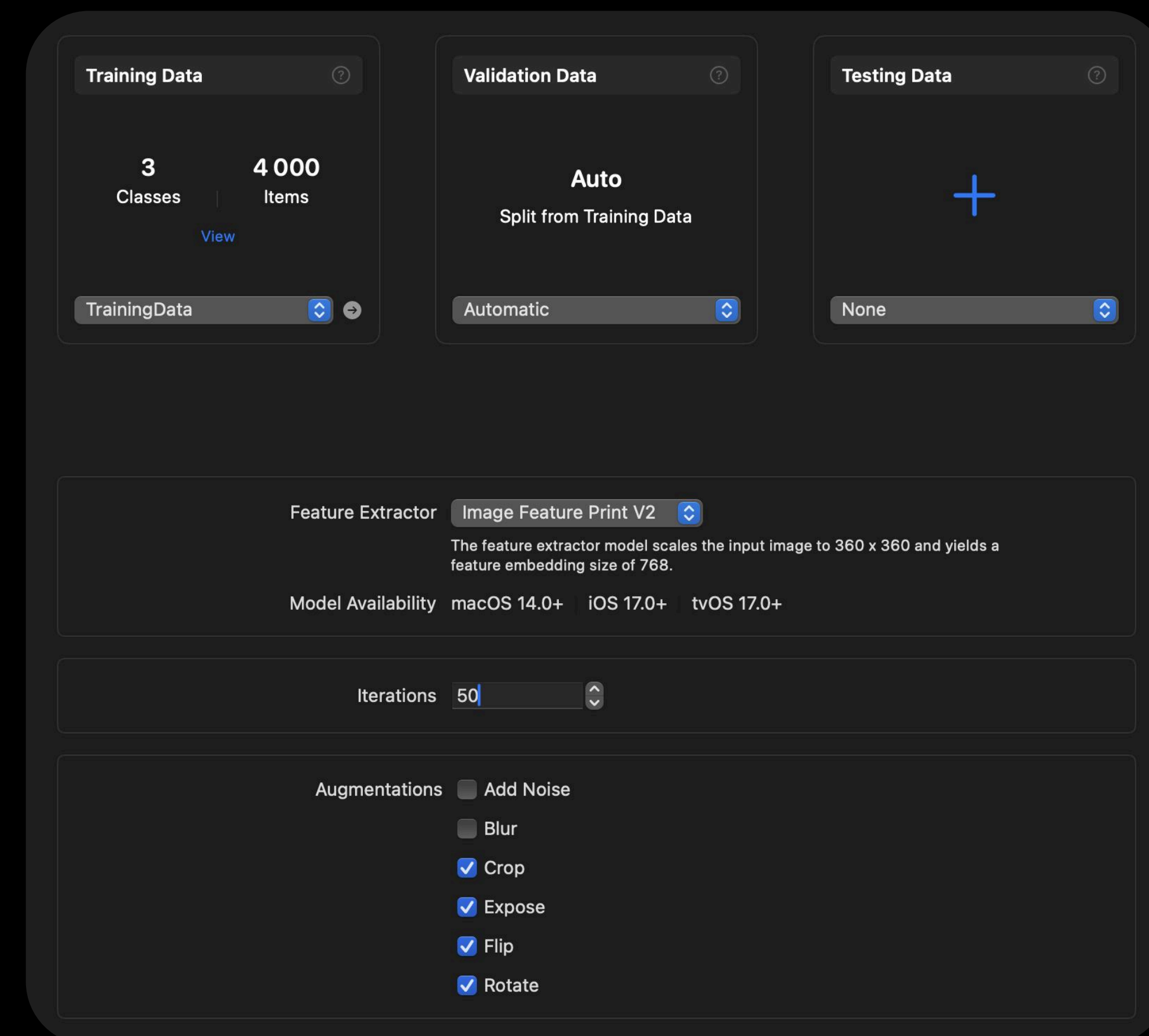
В итоге получаем наш датасет для *training* (на
этих данных модель будет учиться
подстраивать веса)



Шаг 3

Обучение модели в Create ML

1. Загружаем наш датасет в Training Data
2. В Validation Data автоматически выгрузятся ~20% данных из Training Data (это данные, на которых модель проверяется на переобучение, раннюю остановку и тд)
3. Настраиваем параметры:
 - 50 итераций (чем больше тем точнее модель, но и выше риск переобучения)
 - Flip (отражение) – у еды нет “права” и “лева”, поэтому зеркальность не меняет класс
 - Rotate (поворот) – блюдо может быть снято под углом
 - Crop (кадрирование) – часть блюда может не попасть в кадр
 - Expose (яркость/экспозиция) – на разных фото может быть разная яркость и засветы
4. Запускаем обучение (*train*) и ждем



Шаг 4

Анализируем результат

Графики:

- Training Accuracy – точность на данных, по которым модель училась
- Validation Accuracy – точность на данных, которые модель во время обучения не видела (честная оценка обобщения)

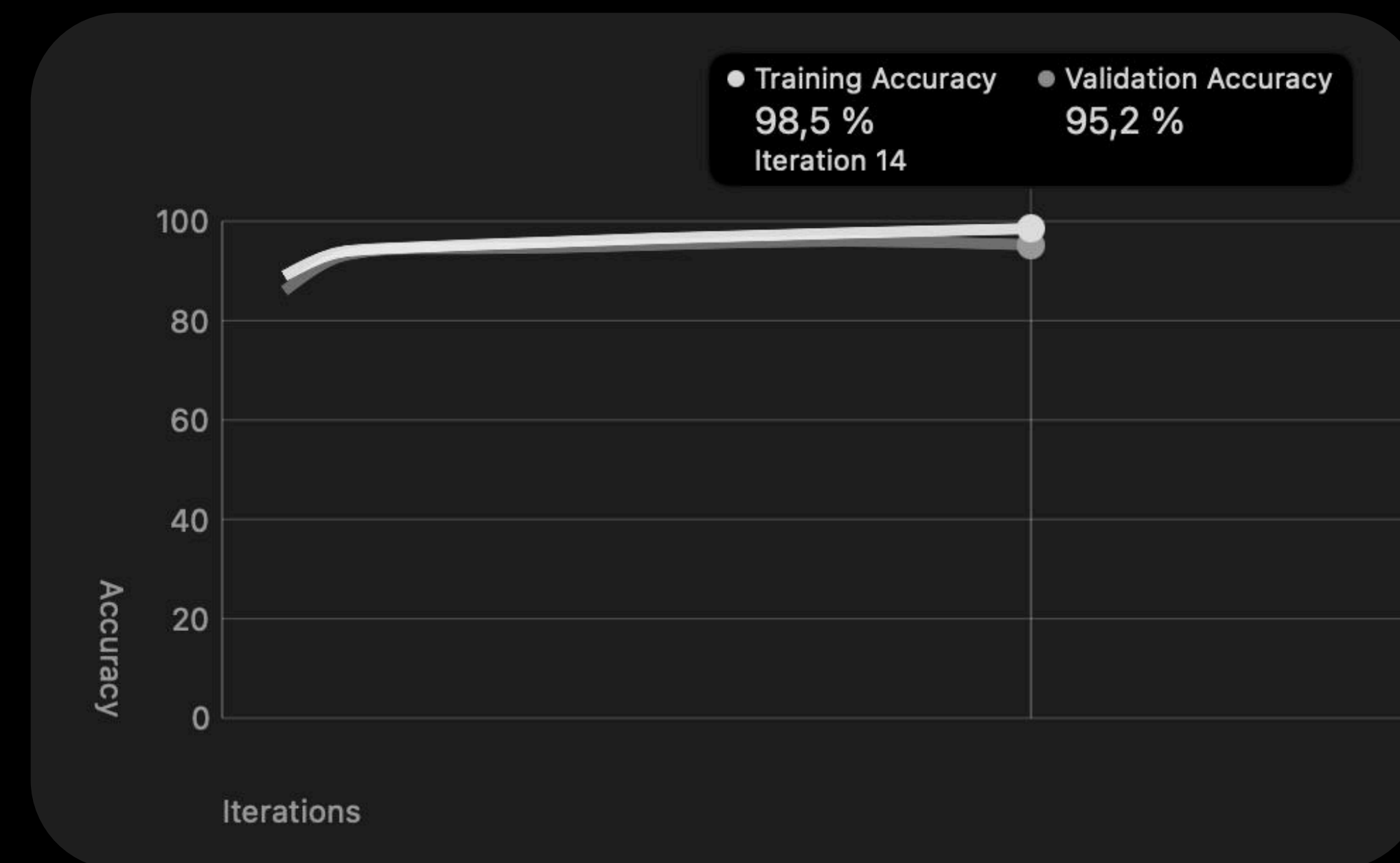
Что можно заметить?

1. Обе линии растут и выходят на плато – модель сошлась, дальше учиться почти нечему
2. Training 98,5%, Validation 95,2% – переобучения нет, модель нормально переносит знания на новые фото (если бы Training был сильно выше Validation (например, 99% vs 70%) – это переобучение: модель «запомнила» обучающую выборку)

Вывод:

Модель обучена и готова к применению в нашем проекте.

Экспортируем ее через *Output* → *Get*



Training Completed

Model converged at 14 iterations

модель остановилась на 14 итерации из 50 – потому что метрики на validation перестали улучшаться (это ранняя остановка)

Шаг 5

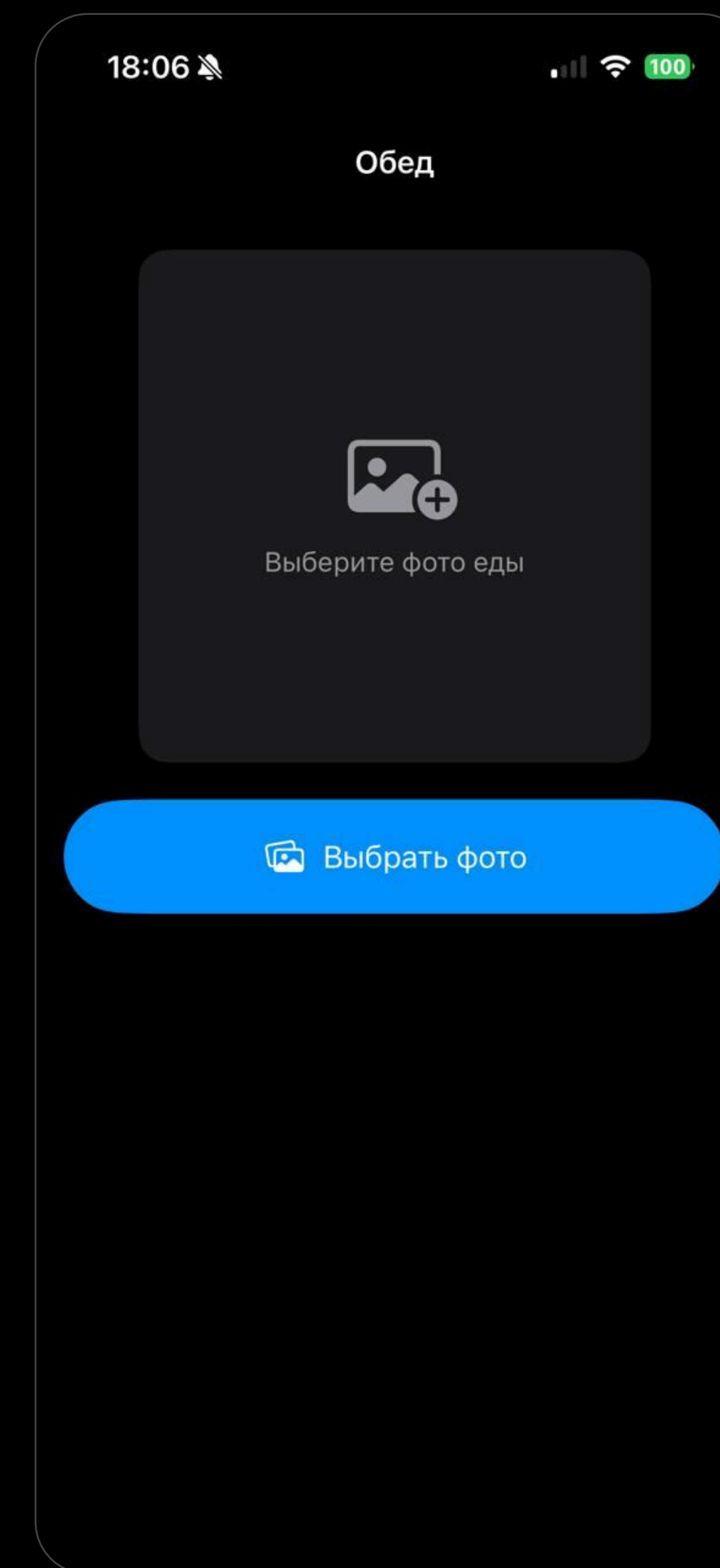
Создаем приложение

Верстаем простой экран на SwiftUI:

- Берем фото из галереи с помощью PhotosPicker (PhotosUI)
- Фотки бывают разных размеров, поэтому скейлим их под один квадрат
- При первом заходе показываем плейсхолдер

Бизнес логика

- Модель отдаёт метку класса и уверенность (confidence)
- Если уверенность меньше 80% – говорим пользователю, что не можем классифицировать блюдо (не вводим в заблуждение)



Шаг 6

Интеграция модели в проект

```
22 private func loadModel() {
23     guard let modelURL = Bundle.main.url(forResource: "FoodClassifier", withExtension: "mlmodelc")
24         ?? Bundle.main.url(forResource: "FoodClassifier", withExtension: "mlmodel") else { return }
25     do {
26         let config = MLModelConfiguration()
27         let mlModel = try MLModel(contentsOf: modelURL, configuration: config)
28         visionModel = try VNCoreMLModel(for: mlModel)
29     } catch {
30         print("Ошибка загрузки модели: \(error)")
31     }
32 }
33
34 func classify(image: UIImage) async -> (classLabel: String, confidence: Float)? {
35     guard let ciImage = CIImage(image: image),
36         let model = visionModel else { return nil }
37
38     return await withCheckedContinuation { continuation in
39         let request = VNCoreMLRequest(model: model) { request, error in
40             if let error = error {
41                 print("Ошибка классификации: \(error)")
42                 continuation.resume(returning: nil)
43                 return
44             }
45             guard let results = request.results as? [VNClassificationObservation],
46                 let top = results.first else {
47                 continuation.resume(returning: nil)
48                 return
49             }
50             continuation.resume(returning: (top.identifier.lowercased(), top.confidence))
51         }
52         request.imageCropAndScaleOption = .centerCrop
53         let handler = VNImageRequestHandler(ciImage: ciImage, options: [:])
54         do {
55             try handler.perform([request])
56         } catch {
57             continuation.resume(returning: nil)
58         }
59     }
```


Шаг 7

Смотрим на результат

18:21

100

Обед



Выбрать фото

✕

Pizza

97% уверенность

18:21

100

Обед



Выбрать фото

✕

Sushi

99% уверенность

18:21

100

Обед



Выбрать фото

✕

?

Не уверены, что это пицца, суши или бургер

52% — слишком низкая уверенность или другой класс

18:19

100

Обед



Выбрать фото

✕

Pizza

99% уверенность

**Но кажется чего то не
хватает?**

Подсчёта калорий!

В чём могут быть проблемы и как их решать?

- Учить на данных с привязкой калорий к порции
- Показывать диапазон или допускать погрешность $\pm 100\text{--}200$ ккал

1 Размер порции на фото неочевиден

3 Мало данных с разметкой калорий

Объединять датасеты, аугментации

2 Разный состав при одном виде блюда

4 Дисбаланс классов (много «средних», мало крайних)

Ограничивать большие классы при сборке датасета

- Больше разнообразных данных
- Классы-диапазоны вместо точного числа

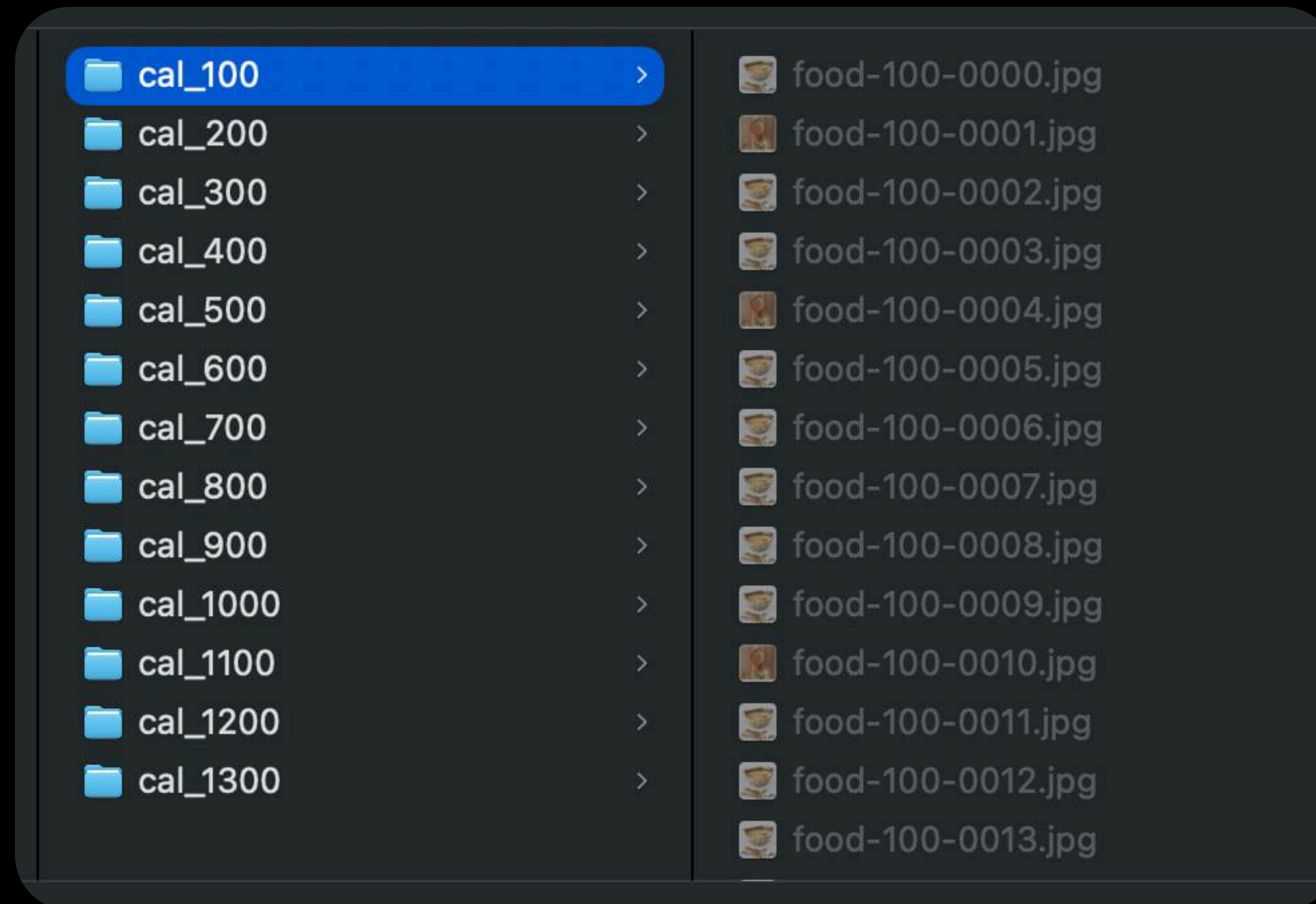
Итог

**Калории по фото — задача сложнее и
неоднозначнее, чем «что за блюдо»**

**упростим её до классификации по диапазонам и
явного допущения погрешность. Пользователю
будем показывать диапазон или округлённое
значение**

Шаг 1

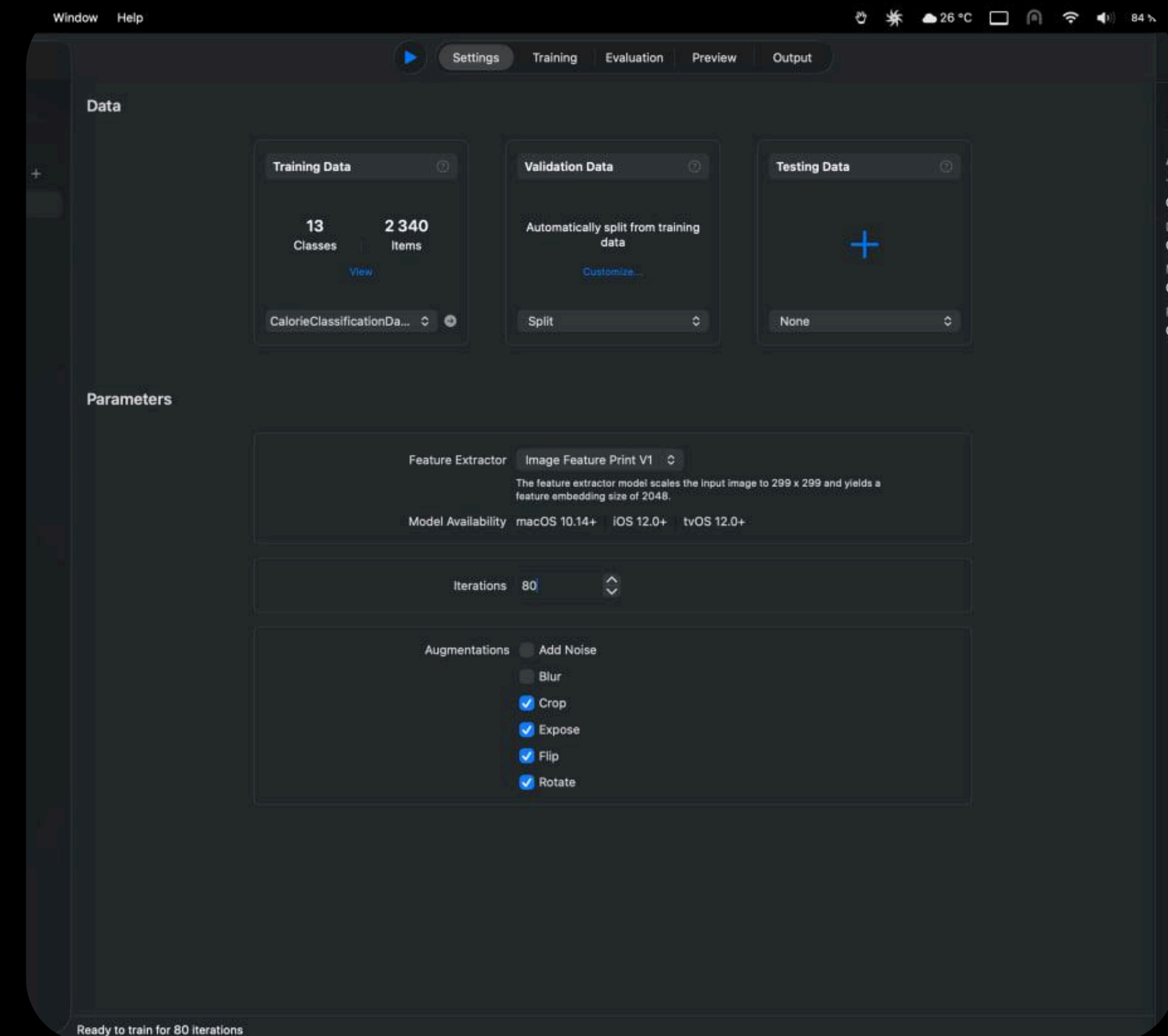
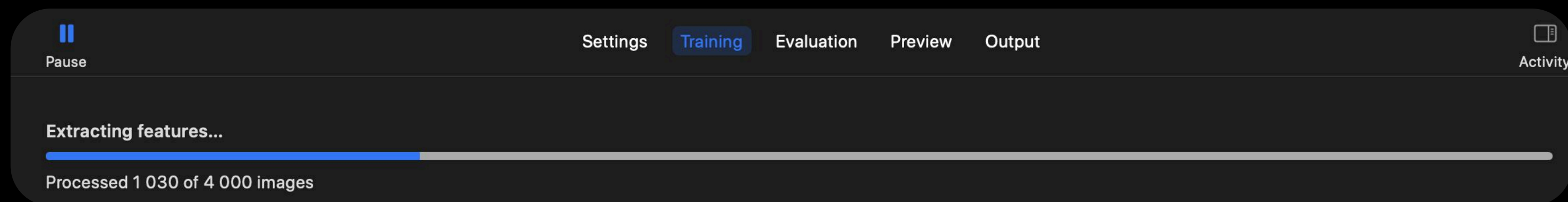
Подготавливаем данные



Шаг 2

Обучение модели в Create ML

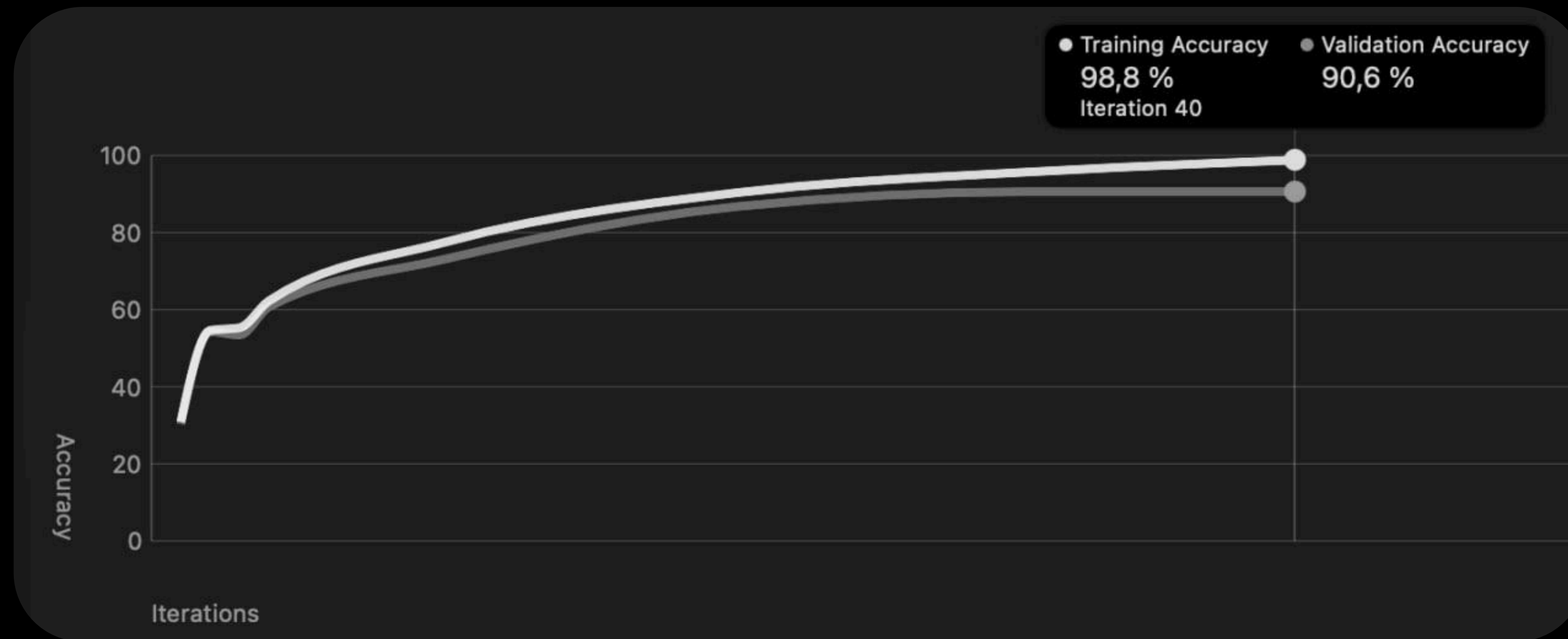
1. Загружаем наш датасет в Training Data
2. В Validation Data автоматически выгрузятся ~20% данных из Training Data (это данные, на которых модель проверяется на переобучение, раннюю остановку и тд)
3. Ставим такие же параметры
4. Запускаем обучение (*train*) и ждем



Шаг 4



Анализируем результат



Вывод:

Модель обучена и готова к применению в нашем проекте.

Экспортируем ее через *Output* → *Get* и интегрируем в проект

Шаг 7

Смотрим на результат

18:57

Обед



Выбрать фото

Pizza

99% уверенность

🔥 900 ккал

на порцию

18:57

Обед



Выбрать фото

Sushi

99% уверенность

🔥 400 ккал

на порцию

18:57

Обед



Выбрать фото

?

Не уверены, что это пицца, суши или бургер

54% — слишком низкая уверенность или другой класс

🔥 —

Калории не показываем для неизвестных блюд

Спасибо за внимание